

Notas de Aula: Abstração de Dados e Estruturas Lineares em C

Prof. Me. Luis Vinicius Costa Silva

16 de março de 2026

Sumário

1	Objetivos da Aula	2
2	Problema Motivador	2
3	Abstração de Dados – Pilha Exemplo	2
4	Tipos Abstratos de Dados (TAD)	2
5	Ponteiros em C	2
6	Exemplo – Troca com Ponteiros	3
7	Exemplo TAD – struct student	3
8	Pilha – Conceito e Exemplo Estático	4
9	Fila – Conceito e Exemplo Circular	4
10	Lista Simplesmente Encadeada	4
11	Lista Duplamente Encadeada	5
12	Árvore Binária de Busca (BST)	5
13	Exercícios Sugeridos	6

1 Objetivos da Aula

Resumo: Apresentar os conceitos de TADs (Tipo Abstrato de Dados) e estruturas lineares como pilhas, filas, listas e árvores, mostrando como representá-los e manipulá-los em C.

Observações: - Destacar a diferença entre conceito e implementação. - Explicar que a abstração facilita manutenção e reutilização.

Código / exemplo: Não aplicável aqui.

2 Problema Motivador

Resumo: Sistema de tarefas com inserção, remoção, histórico e prioridades.

Observações: - Perguntar aos alunos: qual estrutura usar para histórico? - Mostrar que o tipo de TAD depende do problema (LIFO ou FIFO).

Código / exemplo: Não aplicável aqui.

3 Abstração de Dados – Pilha Exemplo

Resumo: Pilha (Stack) é LIFO, esconde detalhes da implementação.

Observações: - Importante enfatizar modularidade. - Mostrar interface mínima: push, pop, top.

Código:

```
1 typedef struct {
2     int data[100];
3     int top;
4 } Stack;
5
6 void push(Stack *s, int value);
7 int pop(Stack *s);
```

4 Tipos Abstratos de Dados (TAD)

Resumo: TAD = valores + operações; define "contrato" da estrutura.

Observações: - Destacar que pilha e fila são TADs clássicos. - Pergunta rápida: qual TAD serve para histórico de ações? Resposta: Pilha (LIFO).

Código / exemplo: Não aplicável aqui.

5 Ponteiros em C

Resumo: Ponteiros armazenam endereços. Operadores: & e *

Observações: - Fundamental para listas e árvores dinâmicas. - Explicar diferença entre passagem por valor e por referência.

Código:

```
1 int a = 90;
2 int *mema = &a;           // mema guarda o endereço de a
```

```
3 printf("%d\n", *mema); // imprime 90
4 *mema = 100;           // altera a a agora vale 100
```

6 Exemplo – Troca com Ponteiros

Resumo: Função swap por valor x por referência

Observações: - Mostrar erro comum: trocar valores sem ponteiros não altera variável externa.

Código:

```
1 // errado
2 void troca_errada(int i, int j) {
3     int temp = i;
4     i = j;
5     j = temp;
6 }
7
8 // correto
9 void troca_certa(int *i, int *j) {
10    int temp = *i;
11    *i = *j;
12    *j = temp;
13 }
```

7 Exemplo TAD – struct student

Resumo: Estrutura simples com nome e nota

Observações: - Incentivar alunos a criar TADs mais complexos (ex.: múltiplas notas, média ponderada).

Código:

```
1 struct student {
2     char *name;
3     float marks;
4 };
5
6 int main() {
7     struct student student1;
8     student1.name = "Aline";
9     student1.marks = 5.6;
10
11     printf("Nome: %s\n", student1.name);
12     printf("Nota: %.1f\n", student1.marks);
13     return 0;
14 }
```

8 Pilha – Conceito e Exemplo Estático

Resumo: Pilha LIFO; operações push, pop.

Observações: - Explicar overflow e underflow. - Mostrar que pilha estática tem limite definido.

Código:

```
1 #define SIZE 100
2 typedef struct {
3     int data[SIZE];
4     int top;
5 } Stack;
6
7 void push(Stack *s, int val){
8     if(s->top < SIZE-1) s->data[++s->top] = val;
9 }
10
11 int pop(Stack *s){
12     return s->top >= 0 ? s->data[s->top--] : -1;
13 }
```

9 Fila – Conceito e Exemplo Circular

Resumo: Fila FIFO; operações enqueue, dequeue.

Observações: - Mostrar vantagem da fila circular sobre fila simples. - Enfatizar gerenciamento de índice front/rear.

Código:

```
1 #define SIZE 5
2 typedef struct {
3     int data[SIZE];
4     int front, rear;
5 } Queue;
6
7 void enqueue(Queue *q, int val){
8     q->data[q->rear] = val;
9     q->rear = (q->rear+1)%SIZE;
10 }
11
12 int dequeue(Queue *q){
13     int val = q->data[q->front];
14     q->front = (q->front+1)%SIZE;
15     return val;
16 }
```

10 Lista Simplesmente Encadeada

Resumo: Nó = valor + ponteiro para próximo. Operações: inserção, remoção, busca.

Observações: - Mostrar que inserção no início é $O(1)$. - Comparar com lista sequencial (vetor).

Código:

```
1 typedef struct Node{
2     int data;
3     struct Node *next;
4 } Node;
5
6 void insert(Node **head, int val){
7     Node *n = (Node*)malloc(sizeof(Node));
8     n->data = val;
9     n->next = *head;
10    *head = n;
11 }
```

11 Lista Duplamente Encadeada

Resumo: Nó com ponteiro para anterior e próximo; permite travessia em ambos sentidos.

Observações: - Remoção pode ser feita sem percorrer toda a lista se ponteiros corretos estiverem definidos.

Código:

```
1 typedef struct Node{
2     int data;
3     struct Node *prev, *next;
4 } Node;
```

12 Árvore Binária de Busca (BST)

Resumo: Estrutura hierárquica, cada nó tem até 2 filhos.

Observações: - Percursos: preorder, inorder, postorder. - Inserção recursiva.

Código:

```
1 typedef struct Node{
2     int data;
3     struct Node *left, *right;
4 } Node;
5
6 Node* insert(Node* root, int val){
7     if(!root){
8         Node* n = (Node*)malloc(sizeof(Node));
9         n->data = val; n->left = n->right = NULL;
10        return n;
11    }
12    if(val < root->data) root->left = insert(root->left, val);
13    else root->right = insert(root->right, val);
14    return root;
15 }
```

```

16
17 void inorder(Node* root){
18     if(root){
19         inorder(root->left);
20         printf("%d ", root->data);
21         inorder(root->right);
22     }
23 }
24
25 int main(){
26     Node* root = NULL;
27     root = insert(root, 20);
28     root = insert(root, 10);
29     root = insert(root, 30);
30
31     printf("Percurso inorder: ");
32     inorder(root);
33     printf("\n");
34     return 0;
35 }

```

13 Exercícios Sugeridos

Resumo: - Simular undo com pilha. - Filas: simular atendimento de clientes. - Listas: inserção, remoção e busca. - Árvores: inserir e percorrer em diferentes ordens.

Observações: - Incentivar desenho de estruturas para melhor compreensão.